

MSI Carcassonne - Konspekt Projektu

Krzysztof Gryboś, Jakub Pietrzak

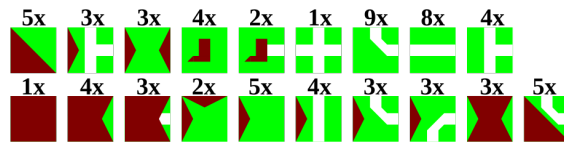
2026

1 Opis problemu

W projekcie zbadamy skuteczność metod planowania strategii w zmodyfikowanej grze *Carcassonne*. Carcassonne to gra strategiczna dla 2-6 graczy, w której gracze kolejno układają płytki, tworząc krajobraz nawiązujący do średniowiecznej Francji. Za tworzenie struktur planszy przyznawane są punkty. Gracz z największą liczbą punktów wygrywa.[7].

W naszym przypadku rozważamy grę o następujących zasadach:

- W grze uczestniczy 2 graczy
- Na początku gry każdy z graczy posiada 8 pionków
- Dostępne są 72 płytki terenu, każda z nich odsłonięta i dostępna do wyboru, wykaz płytek przedstawiono na rysunku 1
- Gracze wykonują ruchy naprzemiennie – ruch polega na dołożeniu płytki do planszy tak żeby rysunek był spójny (typkrawędzi musi się zgadzać) i opcjonalnie dołożeniu jednego pionka na wybranym obszarze na płytce (jeśli na danym obszarze nie znajduje się już inny pionek)
- Za tworzone struktury otrzymywane są punkty. Punkty podliczane są w momencie ukończenia struktury lub po pod koniec gry jeśli nie została ukończona przez całą grę. Punkty otrzymuje gracz, który ma na obszarze największą liczbę pionków.
 - Ukończone miasta – po 2 punkty za kafelek i po 2 punkty za tarcze, nieukończone miasta – po 1 punkt za kafelek i po 1 punkt za tarczę
 - Drogi – po 1 punkt za kafelek
 - Klasztor – 1 punkt i po 1 punkt za każdy graniczący kafelek
 - Pole – po 3 punkty za każde ukończone miasto graniczące z polem
- Wygrywa gracz z największą liczbą punktów po wykorzystaniu wszystkich kafelków



Rysunek 1: Zestaw kart planszy w oryginalnej wersji gry Carcassonne

W porównaniu do oryginalnej gry wprowadzamy następujące modyfikacje:

- Wszystkie płytki są dostępne do wyboru, podczas gdy w oryginalnych zasadach płytki były losowane
- Rozważamy tylko przypadek gry 2 graczy

2 Wstępny przegląd literatury

Monte Carlo Tree Search (MCTS)[1][3] jest jedną z najpopularniejszych metod podejmowania decyzji w grach planszowych z pełną informacją. Algorytm zdobył dużą popularność po sukcesach w grach o dużej przestrzeni stanów, takich jak Go. MCTS buduje drzewo możliwych ruchów na podstawie wielu symulacji rozgrywek i na ich podstawie wybiera najbardziej obiecujące ruchy.

Najczęściej stosowaną wersją MCTS jest UCT (Upper Confidence Bound Applied to Trees)[6], zaproponowane przez Kocsisa i Szepesváriego. Algorytm wykorzystuje wzór równoważący eksplorację nowych ruchów oraz eksploatację ruchów, które wcześniej dawały dobre wyniki. Dzięki temu możliwe jest efektywne przeszukiwanie dużej liczby potencjalnych stanów gry.

W literaturze istnieje wiele rozszerzeń klasycznego UCT. Jednym z najpopularniejszych jest RAVE (Rapid Action Value Estimation)[4], które przyspiesza zbieranie informacji o jakości ruchów. RAVE zakłada, że ruch skuteczny w jednej części symulacji może być również dobry w innych podobnych sytuacjach. Pozwala to szybciej estymować wartość ruchów przy małej liczbie symulacji.

Kolejnym często stosowanym usprawnieniem jest Early Cutoff. W klasycznym MCTS symulacje wykonywane są aż do końca gry, co może być kosztowne obliczeniowo. Early Cutoff polega na wcześniejszym przerywaniu symulacji po określonej liczbie ruchów i ocenieniu planszy za pomocą funkcji heurystycznej.

W grach planszowych często stosuje się również heurystyki wyspecjalizowane dla konkretnej gry. W przypadku Carcassonne możliwe jest ocenianie ruchów na podstawie liczby zdobywanych punktów, kontroli dużych obszarów planszy, blokowania przeciwnika lub efektywnego zarządzania pionkami.

3 Rodzaje algorytmów i ich opis

3.1 UCT (Upper Confidence Bound Applied to Trees)

UCT jest podstawową wersją algorytmu Monte Carlo Tree Search. Algorytm wykonuje wiele symulacji możliwych rozgrywek i na ich podstawie buduje drzewo stanów gry.

Każda iteracja algorytmu składa się z czterech etapów:

1. Selection – wybór najbardziej obiecujących ruchów na podstawie wzoru UCT,
2. Expansion – dodanie nowego węzła do drzewa,
3. Simulation – wykonanie losowej symulacji gry,
4. Backpropagation – aktualizacja statystyk zwycięstw i odwiedzin.

UCT wykorzystuje wzór:

$$UCT = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}$$

gdzie:

- w_i – liczba wygranych dla danego ruchu,
- n_i – liczba odwiedzin danego węzła,
- N – liczba odwiedzin węzła nadrzędnego,
- c – współczynnik eksploracji.

Algorytm równoważy eksplorację nowych ruchów oraz eksploatację ruchów wcześniej uznanych za dobre.

3.2 RAVE (Rapid Action Value Estimation)

RAVE jest rozszerzeniem algorytmu UCT opisanym w literaturze dotyczącej MCTS. W klasycznym UCT ruch oceniany jest wyłącznie na podstawie wyników uzyskanych w konkretnym miejscu drzewa gry.

RAVE wykorzystuje dodatkową informację z całej symulacji. Jeżeli dany ruch prowadził do dobrych wyników w innych częściach symulacji, algorytm szybciej zwiększa jego ocenę również w aktualnym stanie gry.

Pozwala to szybciej uzyskiwać dobre wyniki przy ograniczonej liczbie symulacji.

3.3 Early Cutoff

Early Cutoff polega na wcześniejszym przerywaniu symulacji. W klasycznym MCTS symulacja wykonywana jest aż do zakończenia gry, co może być kosztowne czasowo w przypadku rozbudowanych plansz.

W proponowanym rozwiązaniu symulacja zostanie zatrzymana po określonej liczbie ruchów, a aktualny stan planszy zostanie oceniony funkcją heurystyczną.

Pozwala to:

- zwiększyć liczbę wykonywanych symulacji,
- skrócić czas działania algorytmu,
- ograniczyć wpływ całkowicie losowych końcówek rozgrywki.

3.4 Heurystyka

Dodatkowo zostanie zaimplementowany gracz heurystyczny niewykorzystujący MCTS. Algorytm będzie oceniał ruchy na podstawie reguł specyficznych dla gry Carcassonne.

Przykładowe elementy heurystyki:

- liczba punktów zdobytych po wykonaniu ruchu,
- liczba punktów możliwych do zdobycia w przyszłości,
- blokowanie struktur przeciwnika,
- oszczędzanie pionków,
- zajmowanie dużych obszarów planszy.

Heurystyka będzie wykorzystywana zarówno jako osobny agent porównawczy, jak i potencjalnie do oceny planszy w wariantcie Early Cutoff.

4 Hipotezy badawcze

W projekcie zbadamy prawdziwość następujących hipotez:

- Przy ograniczonym czasie działania (do 1 minuty na ruch) ranking metod od najlepszej do najgorszej będzie następujący:
 1. UCT-ECO
 2. RAVE
 3. UCT
 4. Heurystyka

- Dla mniejszej liczby symulacji – ok. 50 RAVE i UCT-ECO będzie najlepsze. dla większych symulacji – ok. 1000 UCT będzie osiągało podobne wyniki.
- Każdy silnik komputerowy będzie wygrywał z człowiekiem częściej niż przegrywał.
- Algorytmy UCT, UCT-ECO i RAVE będą wygrywały z ludźmi w więcej niż 80% gier.

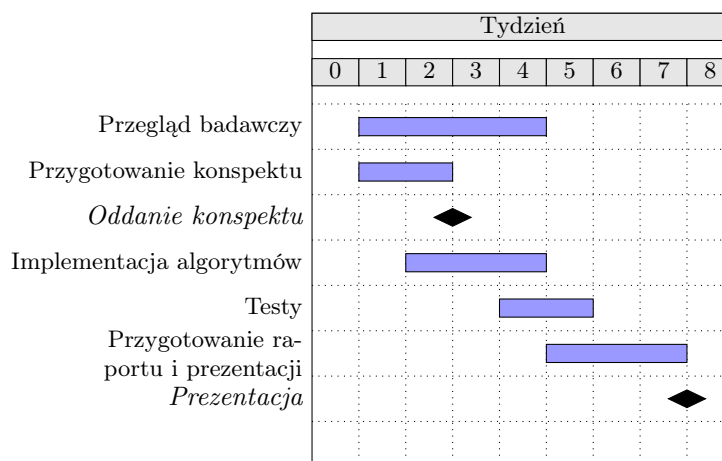
5 Sposób weryfikacji hipotez

Dla zbadaniu skuteczności algorytmów planowane są testy z graczami ludzkimi i turniej między algorytmami. Próby przeciwko ludziom odbędą się na próbce około 5 graczy. Każdy z nich zostanie poproszony o rozegranie 2 rozgrywek z każdym z algorytmów. Zebrane zostaną dane o średniej liczbie wygranych.

Ranking między silnikami komputerowymi zostanie stworzony na podstawie ELO wyliczonego z wyników rozgrywek między silnikami. Planujemy zaimplementować ELO w sformułowaniu Bayesian ELO [2][5]. Ostateczne wyniki zostaną przedstawione w zbiorczej tabeli rankingowej, wraz z graczami ludzkimi.

6 Harmonogram Realizacji

Planowany harmonogram realizacji przedstawiamy na rysunku 2.



Rysunek 2: Diagram Gantta – harmonogram realizacji projektu

7 Ogólny projekt techniczny

Implementacja ma składać się z trzech elementów:

- GUI do grania przeciwko silnikom i do wizualizacji gier – Napisany jako aplikacja webowa w JS
- Silnik gry wybierający kolejny ruch – napisany w języku Rust skompilowanym do WebAssembly
- Backend do zbierania statystyk o grach – proste połączenie z serwerem bazodanowym (opcjonalnie)

Bibliografia

- [1] Bruce Abramson. „The Expected-Outcome Model of Two-Player Games”. W: *Technical Report, Columbia University* (1987).
- [2] Rémi Coulom. „COMPUTING “ELO RATINGS” OF MOVE PATTERNS IN THE GAME OF GO”. W: *ICGA Journal* 30.4 (2007), s. 198–208. DOI: 10.3233/icg-2007-30403.
- [3] Rémi Coulom. „Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search”. W: *Computers and Games*. Berlin, Heidelberg: Springer, 2006, s. 72–83. DOI: 10.1007/11922155_7.
- [4] Sylvain Gelly i David Silver. „Combining online and offline knowledge in UCT”. W: *Proceedings of the 24th International Conference on Machine Learning*. ICML '07. Corvalis, Oregon, USA: Association for Computing Machinery, 2007, s. 273–280. ISBN: 9781595937933. DOI: 10.1145/1273496.1273531. URL: <https://doi.org/10.1145/1273496.1273531>.
- [5] David R. Hunter. „MM algorithms for generalized Bradley-Terry models”. W: *The Annals of Statistics* 32.1 (2004), s. 384–406. DOI: 10.1214/aos/1079120141. URL: <https://doi.org/10.1214/aos/1079120141>.
- [6] Levente Kocsis i Csaba Szepesvári. „Bandit Based Monte-Carlo Planning”. W: *Machine Learning: ECML 2006*. Berlin, Heidelberg: Springer, 2006, s. 282–293. DOI: 10.1007/11871842_29.
- [7] Ober, Matt Harper i CarcassonneCentral Community. *Carcassonne: The Complete Annotated Rules*. Version 6.4. Based on the game by Klaus-Jürgen Wrede. Grud. 2013. URL: https://www.gobblins.net/files/downloads/S-CAR_v6.4.pdf.